AI Engineering

Twelve Foundational Practices

An update and expansion of *AI Engineering: 11 Foundational Practices* (2019)

Carnegie Mellon University
Software Engineering Institute

Authors
Matt Gaston
Eric Heim
Hollen Barmer

[DISTRIBUTION STATEMENT A] Approved for public release and unlimited distribution.

IN 2019, THE SEI PUBLISHED *AI ENGINEERING: 11 FOUNDATIONAL PRACTICES* to help organizations navigate the emerging discipline of building systems that incorporate artificial intelligence (AI). The original 2019 practices have proven durable, but the AI landscape of 2026 is significantly different from 2019 in ways that demand an update.

From an AI Engineering perspective, one of the most significant changes is how organizations acquire and deploy AI capabilities. In 2019, "using AI" almost always meant building a machine learning (ML) model: curate data, select a learning algorithm, train the model, and deploy. Today, AI capabilities are realized through training bespoke models, consuming foundation models via an application programming interface (API), fine-tuning open-weight models, or assembling compound systems (architectures that orchestrate multiple AI and non-AI components to accomplish tasks). Many current production systems combine several of these approaches.[1] Each path carries different implications for data management, security, testing, and operational control.

AI capabilities have advanced substantially along with the engineering challenges. Foundation models can generate fluent text, analyze images, and reason across tasks, but they also generate plausible but incorrect output, behave inconsistently, and resist static testing.[2] Many practical problems remain hard: models drift as data shifts, development performance degrades in production, and AI systems can fail in unexpected ways due to gaps in both reliability and robustness.[3] Agentic systems—AI systems that plan, use tools, and take actions autonomously—introduce further challenges around multi-step reasoning, tool-use safety, and structured human oversight.

Sound AI Engineering is what separates organizations delivering sustained value from those that cannot move beyond prototypes. Rigorous evaluation, systematic data management, appropriate human oversight, and architectures designed to accommodate change are the engineering disciplines that turn AI investments into sustained results.

AI components and systems continue to demand engineering approaches beyond traditional software engineering practices.[4] Real-world deployments show challenges at every stage of the AI system lifecycle, with no stage having fully mature solutions.[5] AI systems differ from traditional software in fundamental ways: behavior emerges from data rather than specification, production performance shifts as distributions change, and components are entangled, complicating modularity. Because AI models learn approximate relationships from their training data rather than implementing exact rules, they can produce unreliable results when they encounter inputs that differ from the data they were trained on. Software engineering is a useful foundation but is not sufficient.

One practice notably absent from the original 11 is evaluation. In 2019, evaluation typically meant accuracy on a held-out test set, and it was reasonable to treat it as a step within other practices. Today, evaluation encompasses drift detection, measuring imbalance, tracking the generation of invalid output, safety testing, and continuous production monitoring. In 2026, evaluation is promoted to a standalone practice.

Sound engineering is what makes rapid deployment possible; without the AI Engineering practices described here, speed creates risk rather than advantage.

Commitments to accelerating AI adoption have matured in parallel. The National Institute of Standards and Technology (NIST) AI Risk Management Framework (RMF) and its generative AI extension provide actionable structure for managing AI risk.[2,6] White House Executive Order 14179[7] signaled national commitment to removing barriers to AI adoption. The U.S. Department of War's (DoW's) January 2026 AI acceleration memorandum[8] directs that vendors deploy the latest models within 30 days of public release. Sound engineering is what makes rapid deployment possible; without the AI Engineering practices described here, speed creates risk rather than advantage.

This update preserves the 2019 document's strengths, brevity, technology agnosticism, and practical orientation while reflecting seven years of research, development, and operational experience across the field. The 12 practices that follow are the engineering foundation for turning AI capability into mission value.

[DISTRIBUTION STATEMENT A] Approved for public release and unlimited distribution.

1. **Ensure you have a problem that should be solved by AI, and select the right approach for the problem and context.**

   AI is not appropriate for every problem. Problems that require fully specified behavior, demand complete auditability of decisions, or have a low tolerance for error may be better addressed through conventional software or simpler analytical methods. For example, validating a shipping address against a postal database is a poor fit for a probabilistic model, while classifying novel images across thousands of categories is a poor fit for hand-coded rules. The NIST AI RMF "Map" function provides a structured approach for scoping whether and how AI fits a given use case.[6] This assessment remains the first and most important engineering decision and should be revisited as requirements, capabilities, and operational context evolve.

   Equally critical is selecting the right approach. The range of options described above, from bespoke training to compound systems, carry substantially different implications for cost, data requirements, control, security, and maintainability. Evaluate the total cost of ownership, including inference, evaluation, monitoring, and ongoing oversight, not just the cost to build the prototype.[4] Similarly, the problem and context dictate other needs: a predictive-maintenance system may require data unique to certain machinery wear patterns, an on-premises medical imaging diagnostic system may need to run inside a secure network with documented change-control procedures, and the size, weight, and power constraints of a drone platform may necessitate the use of a small-scale computer vision model rather than a large one that resides at the top of benchmark leaderboards.

2. **Build cross-functional teams with the domain expertise, engineering depth, and organizational support to develop and operate AI systems.**

   Effective AI Engineering depends on integration across disciplines. Teams need expertise in AI system behavior, in managing the data that shapes system performance, and in evaluating whether system outputs are correct and appropriate in context. Treating AI as a plug-in component that a separate team develops and hands off rarely succeeds. Required capabilities have grown as the field has matured: teams now need expertise that spans system engineering, platform engineering, evaluation design, and prompt engineering alongside traditional software and data skills. What matters is not that every capability is represented by a dedicated role, but that the people building, evaluating, and operating AI systems bring sufficient breadth to reason about the full system lifecycle.[4]

   Domain expertise deserves particular emphasis. Domain experts are essential for defining what correct means in context, identifying failure modes that purely technical evaluation overlooks, and judging whether a system is safe to operate. A data cascade is a compounding negative effect that occurs when data issues in early stages propagate through the system lifecycle. In high-stakes AI systems, 43% of data cascades have been traced to inadequate domain expertise.[9] As AI systems take on higher-stakes tasks, the cost of operating without adequate domain knowledge increases. Organizations should also invest in AI literacy across the teams that interact with, manage, or depend on AI systems, because teams that understand the capabilities and limitations make better decisions about their use. Finally, organizations should have not only AI literacy, but also the ability to connect AI capabilities to specific workflows and the organizational support to change existing workflows if AI adds value.[10]

3. **Take your data seriously across the full system lifecycle.**

   The data that shapes AI system behavior extends well beyond training sets to include retrieval information, fine-tuning datasets, evaluation benchmarks, and context provided at inference time. Regardless of how data is used, the quality, relevance, and representativeness of this data have a direct impact on system performance, and data work remains one of the most underestimated aspects of AI Engineering. Research in data-centric AI reinforces that systematically improving data is often more effective than improving models.[11]

   Several data challenges deserve specific attention. Data provenance is an engineering concern that applies across all AI approaches. Provenance includes where data comes from, how it is licensed, and whether it is representative of the deployment context. Drift is a persistent source of degraded performance that requires different detection and mitigation strategies depending on the nature of the change. Changes include drift in the input distributions (data drift), the relationship between inputs and outputs (concept drift), or the broader environmental conditions.

   For foundation model applications, the relevant data concerns include retrieval quality, prompt design, and in-context examples, not only training datasets.[12] The use of synthetic data is becoming more common but introduces new quality and representativeness concerns that practitioners have not yet fully addressed.[13] Finally, evaluation data curation and benchmark design demand engineering rigor. Applying systematic, reproducible engineering practices to data curation and benchmark construction is essential for reliability and ensuring evaluations reflect downstream real-world performance.

4. **Select AI approaches based on the requirements of the problem, not the appeal of the technology.**

   AI approaches are often selected because they generate attention and excitement but are poorly matched to the problem trying to be solved. Selecting the right AI approach means choosing among rules engines, classical ML models, specialized models, foundation models consumed via API, fine-tuned open-weight models, compound AI systems that combine several of these, and agentic systems that plan and

execute multi-step tasks autonomously. [1] Each option differs across dimensions that matter in practice: cost, latency, interpretability, controllability, data requirements, and operational complexity. The simplest approach that meets performance and governance requirements is often the best one.[4, 6] Establish interpretability requirements early because some approaches are more explainable than others, and retrofitting interpretability is often impractical.

Compound and agentic systems, which orchestrate multiple AI and non-AI components, are increasingly common.[14] These architectures can offer better cost-performance tradeoffs over relying on a single model, but they add engineering complexity in orchestration, testing, failure handling, and oversight. The more autonomy a system has, the more important it becomes to design for observability and human review at critical decision points. Plan for the likelihood that the best available approach will change. Foundation model capabilities shift at least quarterly, and designing for substitutability (see Practice 9) is an architectural response to this reality.

5. **Secure AI systems across the full threat surface, from training-time attacks to runtime exploitation.**

   AI systems expand the attack surface beyond traditional software, requiring dedicated security attention. Classical threats like training data poisoning, adversarial inputs, and model theft remain relevant, but the threat landscape has grown considerably. Systems that consume third-party models or components face supply chain risks, including compromised model weights, poisoned open source packages, and unverified artifacts. The Coalition for Secure AI recommends a defense-in-depth strategy that includes data provenance tracking, model signing, and AI-specific security assessments.[15] Know the provenance of every AI component in a system, and have a plan for responding when a component is found to be compromised.

   Systems built on foundation models introduce additional attack categories. The Open Worldwide Application Security Project (OWASP) GenAI Security Project identified prompt injection, system prompt leakage, excessive agency, and vector and embedding weaknesses as leading risks.[16] Agentic systems amplify these concerns by granting AI components operational authority (the ability to execute actions).[17] Red teaming has emerged as a practice for identifying these risks, although it still has limitations. Adopt red teaming as one component of a broader security program that incorporates threat modeling, designed-in security, continuous monitoring, and incident response.[18, 19]

6. **Implement traceability and version control across all AI system artifacts to enable recovery, debugging, and accountability.**

   AI systems are sensitive to changes in their dependencies, and the set of artifacts to track has grown beyond models and training data. Depending on the approach, tracked artifacts may include evaluation datasets, prompts, retrieval configurations, guardrail settings, fine-tuning data, tool definitions, and specific versions of third-party models or APIs. When a foundation model is updated, behavior can change without any modification to system code. Systematic versioning and experiment tracking across these artifacts are baseline engineering practices, not optional infrastructure.[20]

   Traceability is necessary for debugging, maintainability, and accountability. When a compound AI system produces an unexpected result, understanding which component caused the regression requires the ability to trace through the full system architecture. For agentic systems, traceability is even more important to reconstruct what context the system had, what decisions it made, what tools it invoked, and with what permissions.[19] More broadly, documenting why a system was configured in a specific way, what data informed those decisions, and what tradeoffs were made makes debugging faster, onboarding easier, and audits traceable. Design AI systems for auditability from the start rather than retrofitting after deployment.[21]

7. **Design human-AI interaction and oversight appropriate to the system's risk level and operating context.**

   Users remain the primary feedback mechanism for AI systems, and the design of human-AI interaction is a critical engineering concern. The right oversight pattern depends on the stakes, the operational tempo, and the nature of the AI system. Some applications warrant human review of every output; others require monitoring at a statistical level; and some operate autonomously with only periodic review. Effective oversight balances operational efficiency with appropriate control. Systems that need to respond in milliseconds require monitoring rather than approval gates, while high-stakes systems benefit from structured review at critical decision points.[22] Where human oversight alone is insufficient (due to speed, scale, or complacency) it should be complemented by technical safeguards like automated constraint enforcement, output filtering, and anomaly-triggered circuit breakers. The effectiveness of oversight mechanisms should be measured, not assumed. Tracking metrics like override rates and time-on-task can reveal when human review has become insufficient.

   A well-documented challenge is automation complacency: human overseers tend to accept AI recommendations without sufficient scrutiny, particularly as system performance improves and trust builds over time.[23] Recent work identifies a deeper pattern—cognitive surrender—where users both defer to AI and relinquish their own reasoning

[DISTRIBUTION STATEMENT A] Approved for public release and unlimited distribution.

process.[24] Placing a human in the loop does not guarantee effective oversight. Interaction design must actively support engagement, giving overseers the context and tools to evaluate outputs critically rather than accept them reflexively. For agentic systems, the design challenge requires defining explicit boundaries around what the system can do independently, what requires human approval, and what is prohibited, along with structured escalation mechanisms for when the system encounters situations outside its defined scope and authority.[19]

8. **Treat model uncertainty as a first-class engineering concern.**

   Many AI models produce outputs with inherent uncertainty, but the nature of the uncertainty varies by approach and can be difficult to characterize precisely. Classical models produce confidence scores that, when properly calibrated, provide a quantitative basis for trusting or questioning the result. Generative models present a different challenge: they produce fluent, plausible outputs that can obscure factual errors or knowledge gaps. This asymmetry, where incorrect outputs look indistinguishable from correct ones, makes engineering for uncertainty more important and more difficult. A model that confidently cites nonexistent sources or generates plausible but fabricated statistics illustrates why traditional error-handling assumptions break down.

   Uncertainty calibration, or confidence calibration (ensuring that a system's expressed confidence corresponds to its probability of being correct), remains a challenging problem. Gaps in both reliability and robustness contribute to this challenge. AI systems can behave unpredictably when encountering inputs outside the conditions they were trained on.[3] Implement technical methods to quantify and detect low-confidence outputs, and design mechanisms to communicate uncertainty to upstream processes and human users that support good decision making.[25, 26]

   Approaches to model uncertainty span confidence scoring, consistency checks across multiple generations, retrieval-augmented grounding, and pre-generation risk assessment. Which technique to use depends on the application and the stakes. Equally important is how uncertainty is communicated. Explicit, calibrated expressions of uncertainty compared to overconfident claims have shown improved user decision quality.[27] Engineering decisions involve specifying how uncertainty is surfaced using mechanisms such as confidence indicators, qualitative statements, and confidence-triggered escalation and review. For high-stakes circumstances, design systems to present a range of candidate outputs along with their confidence thresholds, return the decision to a human decision maker, or decline to respond when confidence thresholds are not met.[28]

9. **Architect AI systems for modularity and operational resilience.**

   AI systems accumulate hidden dependencies that differ from those in traditional software and are often harder to detect. Components that share data sources, consume each other's output, or rely on the same upstream features can become entangled so that changing one element silently affects behavior elsewhere in the system. This risk intensifies with compound architectures that integrate foundation models, retrieval systems, guardrails, orchestration logic, and business rules. Each integration point is a dependency surface where model updates, API changes, or distribution shifts can propagate unpredictability. To manage this risk, decompose systems into loosely coupled modules with explicit contracts. Define the inputs and outputs for each component, route communication through standardized interfaces, and isolate model selection so that substitution, version updates, and routing decisions can be made independently.[29, 30]

   Operational resilience depends on observability beyond traditional application monitoring. Best practices include tracking model performance over time, detecting data or concept drift, logging predictions with outcomes, and monitoring for safety violations.[31, 32, 33] Infrastructure should make retraining (for traditional machine learning models or fine tuning of foundation models), model replacement, and component updates routine rather than exceptional, with automated detection of when changes are needed, clear promotion and rollback procedures, and reproducible records of component performance. AI systems also accumulate technical debt in novel ways, including entangled data dependencies, undeclared data consumers, and feedback loops that silently shift system behavior.[34, 35]

10. **Plan and budget for continuous change across models, infrastructure, and team capabilities.**

    The lifecycle of an AI system does not end at deployment. Operational costs including continuous monitoring, model retraining, infrastructure scaling, and dependency management can rival or exceed development costs. Third-party dependency management is subject to external deprecation schedules, pricing changes, behavior shifts from model updates, and the need to maintain architectural flexibility for alternative providers.[36, 37, 38] Cumulatively, these dependencies create a sustained cost that organizations must plan for from the outset. For example, when a third-party model provider deprecates an API version or changes pricing, organizations without architectural flexibility face unplanned migration costs and possible loss of capability. Without explicit budgeting and resource allocation, systems can degrade in performance, incur unexpected expenses, or become operationally unsustainable.

    Current AI systems exist in continuous change. Model performance decays as distributions shift, requiring detection mechanisms and retraining processes that operate as routine

production functions.[31, 32, 33] Infrastructure demands grow with usage, and replacing or scaling compute creates recurrent investment cycles. Team skills gaps widen as the field evolves, making continuous upskilling an operational necessity. Managing computational costs at scale (a dedicated discipline for AI cost management sometimes called "FinOps for AI") helps ensure that inference, training, and infrastructure expenses are monitored and optimized as rigorously as other operational expenses.[37]

11. Apply practical safety engineering across the AI system lifecycle.

AI systems can cause harm even in the absence of adversarial attack. Systems that produce confidently wrong outputs in high-stakes contexts, that behave unpredictably on novel inputs, or that take autonomous actions with unintended consequences present concrete safety engineering challenges distinct from security concerns (see Practice 5). Engineering teams should apply established safety engineering techniques (e.g., hazard analysis, failure mode identification, safety cases, and operational safeguards) adapted for the unique characteristics of AI components. Identify safety considerations during design, verify them through targeted testing during development, and continuously monitor them in production.[32]

Practical safety engineering for AI systems starts with identifying how a system can fail and what harm each failure mode can cause. AI model and systems cards document known limitations and intended operating conditions.[39] Runtime monitoring tracks behavioral indicators (e.g., output confidence distributions, refusal rates, flagged content) to detect when a system operates outside its intended use. NIST's AI Risk Management Framework [2, 6] provides structured approaches for mapping safety concerns to engineering controls. Ask practical safety-engineering related questions throughout the system lifecycle:[40, 41] What failure modes have been identified? What safeguards have been put in place? What has been tested? What should be monitored? Who is accountable when something goes wrong?

12. Invest in evaluation as a first-class engineering challenge.

Evaluation determines whether an AI system is working. It indicates whether the data is fit for purpose, whether safety constraints are being met, whether the system is performing as expected, and whether the system is degrading in production. With ever-broader adoption of AI and agentic systems, the evaluation surface continues to expand. An agentic system that reliably completes tasks in testing may behave unpredictably in production when users issue

Looking Ahead

AI Engineering as a discipline is evolving as fast as the technology it supports, and the pace of change is not slowing. Agentic AI systems, which plan, use tools, and take actions autonomously, are the current frontier. These systems challenge nearly every practice simultaneously: problem suitability assessment must account for open-ended tasks, security must address tool-use exploitation and multi-step attack chains, human oversight must define meaningful autonomy boundaries, and evaluation must grapple with emergent behaviors across increasingly longer action sequences. Several of the foundational practices described in this document address agentic concerns directly. As these systems mature and enter broader production use, additional engineering guidance will be necessary.

Other trends will shape AI Engineering in the near term as well. Multi-modal systems that operate across text, image, audio, and video introduce integration and evaluation challenges that single-modality guidance does not fully cover. Inference-time compute scaling, where models expend variable computational effort based on task difficulty, is changing assumptions about cost, latency, and capability. On-device and edge AI continues to grow, pushing the bounds of compute, connectivity, and fleet management and orchestration. Economics are shifting as the cost of inference compute declines (while use increases) and open-weight and open source models improve, changing the build-versus-buy calculus regularly.

Regulatory frameworks like the NIST AI RMF provide tools for managing risk without waiting for prescriptive rules.[2, 6] AI supply chain security is moving from guidance to operational necessity as incidents involving compromised components demonstrate the risks of unverified dependencies in production AI systems.[46] More broadly, governance is shifting from documentation to automation, with organizations embedding compliance checks, audit mechanisms, and policy enforcement directly into development and deployment pipelines.

AI Engineering is a young discipline operating on a fast-moving technology base. The challenges are real: AI systems degrade silently, fail unpredictably, and require sustained investment to operate reliably. Opportunities are equally real. These practices are intended as a framework for engineering decision-making, not as an implementation guide or checklist. While the engineering concerns will persist, specific techniques and tools will evolve as the technology evolves. Organizations that apply sound engineering discipline to AI will build systems that improve decision making, accelerate operations, and solve problems that were previously intractable.

[DISTRIBUTION STATEMENT A] Approved for public release and unlimited distribution.

ambiguous instructions or when tool APIs return unexpected results. Models and systems that pass pre-deployment evaluation can change behavior after deployment based on distribution shifts, user behavior changes, or dependency updates. Evaluation requires intentional infrastructure investments, including curated evaluation datasets, domain-specific benchmarks, automated pipelines, and human judgement workflows for complex or ambiguous tasks.[42]

Evaluation-driven development is an emerging engineering practice where evaluation criteria and test cases are defined before AI systems are built or deployed.[43] These AI tests are often probabilistic and domain-specific, requiring human judgement to define success criteria and interpret results. Continuous evaluation in production is distinct from pre-deployment testing and uses lightweight monitoring that avoids significant latency or costs, detection mechanisms for unanticipated failure modes, and trustworthy alerting mechanisms.[44] A mature evaluation practice allows engineering teams to drive prioritization, resource allocation, and incremental system design decisions.[45]

References

1. Chen, J., J. Ye, and G. Wang. "From Standalone LLMs to Integrated Intelligence: A Survey of Compound AI Systems." *arXiv* preprint arXiv:2506.04565 (June 2025).
2. National Institute of Standards and Technology. "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile" (AI 600-1, 2024).
3. Heim, E., O. Wright, and D. Shriver. "A Guide to Failure in Machine Learning: Reliability and Robustness from Foundations to Practice." *arXiv* preprint arXiv:2503.00563 (2025); Carnegie Mellon University, Software Engineering Institute.
4. Kästner, C. *Machine Learning in Production: From Models to Products.* Cambridge, MA: MIT Press, 2025.
5. Assres, G., G. Bhandari, A. Shalaginov, T.M. Grønli, and G. Ghinea. "State-of-the-Art and Challenges of Engineering ML-Enabled Software Systems in the Deep Learning Era." *ACM Computing Surveys* 57, no. 10 (2025).
6. National Institute of Standards and Technology. "Artificial Intelligence Risk Management Framework" (AI RMF 1.0, 2023).
7. White House. Executive Order 14179: Removing Barriers to American Leadership in Artificial Intelligence. January 23, 2025.
8. United States Department of War. Memorandum for Senior Pentagon Leadership, Commanders of the Combatant Commands, Defense Agency and DoW Field Activity Directors: Artificial Intelligence Strategy for the Department of War. January 9, 2026.
9. Sambasivan, N., S. Kapania, H. Highfill, D. Akrong, P. Paritosh, and L. Aroyo. "Everyone Wants to Do the Model Work, Not the Data Work: Data Cascades in High-Stakes AI." *Proceedings of the CHI Conference* (2021).
10. Singla, A., A. Sukharevsky, L. Yee, M. Chui, B. Hall, and T. Balakrishnan. "The State of AI in 2025: Agents, Innovation, and Transformation." McKinsey & Company, November 5, 2025.
11. Zha, D., Z. P. Bhat, K.H. Lai, F. Yang, Z. Jiang, S. Zhong, and X. Hu. "DataCentric Artificial Intelligence: A Survey." *ACM Computing Surveys* (2025).
12. Xu, Y., Z. Wu, R. Qiao, et al. "Data-Centric AI in the Age of Large Language Models." *EMNLP* (2024).
13. Kapania, S., S. Ballard, A. Kessler, and J. Vaughan. "Examining the Expanding Role of Synthetic Data Throughout the AI Development Pipeline." *ACM FAccT* (2025).
14. Lee, Y., et al. "Compound AI Systems Optimization: A Survey of Methods, Challenges, and Future Directions." *EMNLP* (2025).
15. Coalition for Secure AI (CoSAI). "Establish Risks and Controls for the AI Supply Chain, V 1.0." OASIS Open, 2025.
16. OWASP. "Top 10 for Large Language Model Applications." OWASP Gen AI Security Project, 2024.
17. OWASP. "Top 10 for Agentic Applications for 2026." OWASP Gen AI Security Project, 2025.
18. Feffer, M., A. Sinha, et al. "Red-Teaming for Generative AI: Silver Bullet or Security Theater?" *arXiv* preprint arXiv:2401.15897 (2024).
19. Coalition for Secure AI (CoSAI). "Principles for Secure-by-Design Agentic Systems." OASIS Open, 2025.
20. Eken, B., S. Pallewatta, N. K. Tran, A. Tosun, and M. A. Babar. "A Multivocal Review of MLOps Practices, Challenges and Open Issues." *ACM Computing Surveys* (2025).
21. Ojewale, V., R. Steed, B. Vecchione, A. Birhane, and I. D. Raji. "Towards AI Accountability Infrastructure: Gaps and Opportunities in AI Audit Tooling." *Proceedings of ACM CHI* (2025).
22. Faas, C., et al. "Design Considerations for Human Oversight of AI: Insights from Co-Design Workshops and Work Design Theory." *arXiv* preprint arXiv:2510.19512 (2025).
23. Romeo, G. and D. Conti. "Exploring Automation Bias in Human–AI Collaboration: A Review and Implications for Explainable AI." *AI & Society* (Springer, 2025).
24. Shaw, S. D., and G. Nave. "Thinking—Fast, Slow, and Artificial: How AI Is Reshaping Human Reasoning and the Rise of Cognitive Surrender." Wharton School, University of Pennsylvania, 2026.
25. Geng, J., et al. "A Survey of Confidence Estimation and Calibration in Large Language Models." *Proceedings of NAACL* (2024).
26. Liu, X., et al. "Uncertainty Quantification and Confidence Calibration in Large Language Models: A Survey." *Proceedings of ACM SIGKDD* (KDD '25) (2025).
27. Steyvers, M., and M. A. K. Peters. "Metacognition and Uncertainty Communication in Humans and Large Language Models." *Current Directions in Psychological Science* (2025).
28. Wen, B., et al. "Know Your Limits: A Survey of Abstention in Large Language Models." *Transactions of the Association for Computational Linguistics* 13 (2025).
29. Kandogan, E., et al. "Orchestrating Agents and Data for Enterprise: A Blueprint Architecture for Compound AI." *IEEE International Conference on Data Engineering Workshops* (ICDEW) (2025).
30. Nowaczyk, S. "Architectures for Building Agentic AI." *arXiv* preprint arXiv:2512.09458 (December 2025).
31. Wan, K., Y. Liang, and S. Yoon. "Online Drift Detection with Maximum Concept Discrepancy." *Proceedings of ACM SIGKDD* (KDD '24) (2024).
32. Zhao, L., and Y. Shen. "Proactive Model Adaptation Against Concept Drift for Online Time Series Forecasting." *Proceedings of ACM SIGKDD* (KDD '25) (2025).
33. Hinder, F., V. Vaquet, and B. Hammer. "One or Two Things We Know about Concept Drift: A Survey on Monitoring in Evolving Environments." *Frontiers in Artificial Intelligence* (2024).
34. Recupito, G., et al. "Technical Debt in AI-Enabled Systems: On the Prevalence, Severity, Impact, and Management Strategies." *Journal of Systems and Software* (2024).
35. Moreschini, S., et al. "The Evolution of Technical Debt from DevOps to Generative AI: A Multivocal Literature Review." *Journal of Systems and Software* (2026).
36. Lu, Q., et al. "A Taxonomy of Foundation Model-Based Systems through the Lens of Software Architecture." *Proceedings of IEEE/ACM CAIN* (2024).
37. Lu, Q., et al. "Toward Responsible AI in the Era of Generative AI: A Reference Architecture for Designing Foundation Model-Based Systems." *IEEE Software* 41, no. 6 (2024).
38. Sens, Y., H. Knopp, S. Peldszus, and T. Berger. "A Large-Scale Study of Model Integration in ML-Enabled Software Systems." *Proceedings of IEEE/ACM ICSE* (2025); *arXiv* preprint arXiv:2408.06226.
39. Liu, J., et al. "Automatic Generation of Model and Data Cards: A Step Towards Responsible AI." *Proceedings of NAACL* (2024).
40. Bengio, Y., et al. "International AI Safety Report 2025: Second Key Update: Technical Safeguards and Risk Management." *arXiv* preprint arXiv:2511.19863 (November 2025).
41. Bengio, Y., et al. "International AI Safety Report 2026." *arXiv* preprint arXiv:2602.21012 (February 2026).
42. Naveed, H., S. Barnett, C. Arora, J. Grundy, H. Khalajzadeh, and O. Haggag. "Monitoring Machine Learning Systems: A Multivocal Literature Review." *arXiv* preprint arXiv:2509.14294 (September 2025).
43. Xia, B., Q. Lu, L. Zhu, Z. Xing, D. Zhao, and H. Zhang. "Evaluation-Driven Development and Operations of LLM Agents: A Process Model and Reference Architecture." *arXiv* preprint arXiv:2411.13768v3 (November 2025).
44. Akshathala, S., et al. "Beyond Task Completion: An Assessment Framework for Evaluating Agentic AI Systems." *Proceedings of AGENT '26* (2026).
45. Mohammadi, M., Y. Li, J. Lo, and W. Yip. "Evaluation and Benchmarking of LLM Agents: A Survey." *Proceedings of ACM SIGKDD* (KDD '25) (2025).
46. National Security Agency, et al. "AI/ML Supply Chain Risks and Mitigations." *Joint Cybersecurity Information Sheet* (March 2026).

About the SEI

The Software Engineering Institute (SEI) advances software as a strategic advantage for national security through research, development, and deployment of tools, technologies, and practices in software engineering, artificial intelligence (AI), cyber, and acquisition transformation. We serve the nation as a federally funded research and development center (FFRDC) sponsored by the U.S. Department of War (DoW).

Contact Us

CARNEGIE MELLON UNIVERSITY
SOFTWARE ENGINEERING INSTITUTE
4500 FIFTH AVENUE; PITTSBURGH, PA 15213-2612

sei.cmu.edu
412.268.5800 | 888.201.4479
info@sei.cmu.edu

Copyright 2026 Carnegie Mellon University.

This material is based upon work funded and supported by the Department of War under Air Force Contract Nos. FA8702-15-D-0002, and FA870225DB003 with Carnegie Mellon University for the operation of the Software Engineering Institute, a federally funded research and development center.

The opinions, findings, conclusions, and/or recommendations contained in this material are those of the author(s) and should not be construed as an official US Government position, policy, or decision, unless designated by other documentation.

References herein to any specific entity, product, process, or service by trade name, trademark, manufacturer, or otherwise, does not necessarily constitute or imply its endorsement, recommendation, or favoring by Carnegie Mellon University or its Software Engineering Institute nor of Carnegie Mellon University - Software Engineering Institute by any such named or represented entity.

NO WARRANTY. THIS CARNEGIE MELLON UNIVERSITY AND SOFTWARE ENGINEERING INSTITUTE MATERIAL IS FURNISHED ON AN "AS-IS" BASIS. CARNEGIE MELLON UNIVERSITY MAKES NO WARRANTIES OF ANY KIND, EITHER EXPRESSED OR IMPLIED, AS TO ANY MATTER INCLUDING, BUT NOT LIMITED TO, WARRANTY OF FITNESS FOR PURPOSE OR MERCHANTABILITY, EXCLUSIVITY, OR RESULTS OBTAINED FROM USE OF THE MATERIAL. CARNEGIE MELLON UNIVERSITY DOES NOT MAKE ANY WARRANTY OF ANY KIND WITH RESPECT TO FREEDOM FROM PATENT, TRADEMARK, OR COPYRIGHT INFRINGEMENT.

[DISTRIBUTION STATEMENT A] This material has been approved for public release and unlimited distribution. Please see Copyright notice for non-US Government use and distribution.

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License (https://creativecommons.org/licenses/by-nc/4.0/). Requests for permission for non-licensed uses should be directed to the Software Engineering Institute at permission@sei.cmu.edu.

This work product was created in part using generative AI.

DM26-0412

©2026 Carnegie Mellon University | 6667 | C 04.20.2026 | S 04.22.2026